

Dinner Poll – Project Recap

1. Overall Architecture

- Next.js 16 with the App Router (app/ directory).
- One main client page for the poll (app/page.tsx).
- API routes for voting and results:
- POST /api/vote
- GET /api/results/[roundId]
- A private admin page at /admin for live results.
- PostgreSQL (Prisma Postgres) as the data store, accessed via the pg client.

2. Data Model (PostgreSQL)

Single table: votes

```
CREATE TABLE IF NOT EXISTS votes (  
  id uuid PRIMARY KEY DEFAULT gen_random_uuid(),  
  round_id text NOT NULL,  
  restaurant_id text NOT NULL,  
  user_id text NOT NULL,  
  score integer NOT NULL,  
  created_at timestamptz DEFAULT now(),  
  updated_at timestamptz DEFAULT now(),  
  UNIQUE (round_id, restaurant_id, user_id)  
);
```

Key ideas:

- One row per (round, restaurant, user).
- score is 0, 1, or 2 (No / OK / Top choice).
- UNIQUE ensures a user can only have one vote per restaurant per round (but can change it, via UPDATE).

3. Backend: lib/db.ts using pg

- Uses DATABASE_URL (Prisma Postgres) and pg.Pool.
- ensureSchema():
- Creates the votes table if it doesn't exist.
- In production (VERCEL_ENV === "production"), truncates the votes table on startup so each new deploy starts with a fresh poll.
- upsertVote():
- INSERT ... ON CONFLICT (round_id, restaurant_id, user_id) DO UPDATE SET score = EXCLUDED.score.
- getRoundResults(roundId):
- SELECT restaurant_id, SUM(score) AS total_score, COUNT(*) AS votes_count
- GROUP BY restaurant_id
- ORDER BY total_score DESC

This teaches:

- How to structure a simple relational model.
- How to use ON CONFLICT for idempotent writes (upsert pattern).
- How to aggregate results (SUM + COUNT) directly in SQL.

4. API Routes (Next.js App Router)

a) app/api/vote/route.ts

- Uses NextRequest and NextResponse.
- Validates payload (roundId, restaurantId, userId, score).

- Calls `upsertVote()`.

Core pattern:

```
export async function POST(req: NextRequest) {
  const { roundId, restaurantId, userId, score } = await req.json();
  // validate
  await upsertVote({ roundId, restaurantId, userId, score });
  return NextResponse.json({ ok: true });
}
```

b) `app/api/results/[roundId]/route.ts`

- Correctly typed for Next 16:

```
export async function GET(
  _req: NextRequest,
  { params }: { params: Promise<{ roundId: string }> }
) {
  const { roundId } = await params;
  const results = await getRoundResults(roundId);
  return NextResponse.json({ results });
}
```

Lessons:

- Correct typing of route handlers for the App Router.
- Handling dynamic route params with `{ params: Promise<{ roundId: string }> }`.
- Separation of concerns: API does no UI work, just returns JSON.

5. Frontend: Main Poll Page (`app/page.tsx`)

Key React concepts used:

- `useState` for:
 - votes (per restaurant)
 - `showResults` (whether user clicked “See results”)
 - `nickname`
 - `joined` (has the user joined the poll?)
 - `userId` (random persistent ID stored in `localStorage`)
 - `results` (aggregated data from the API)
 - loading states
 - `isClosed` and `closingText/timeLeft` (driven by `pollClosesAt`)
- `useEffect`:
 - To generate and persist `userId` in `localStorage`.
 - To compute `isClosed` and a live countdown (`timeLeft`) on the client only.
- Hydration-safe date handling:
 - Avoids using `toLocaleString()` directly in SSR markup.
 - Uses client-side `useEffect` to compute display strings for dates.
- Fetch calls from client to API:
 - On slider change → `POST /api/vote`.
 - On “See results” → `GET /api/results/round-1`.

UX logic:

- User must join (`nickname` + button) before voting (gates interactions).
- While poll is open:
 - Users can vote but do NOT see live ranking.

- When poll is closed:
- “See final results” button fetches and displays final ranking.
- Admin sees live results via /admin.

6. RestaurantCard Component

Key points:

- Receives restaurant (data), value, onChange, and disabled.
- Shows:
 - Name + location + tags.
 - Short description.
 - Slider input (0, 1, 2) mapped to labels (“No / OK / Top choice”).
 - Optional expanded area with images and specialties, toggled with a button.
- Uses internal useState for expanded toggle.
- Demonstrates:
 - Controlled component (slider) controlled by parent state.
 - Props for data vs. local UI state for presentation.
 - Conditional rendering (expanded details, disabled state).

7. Admin Page (/admin)

- Client-side page that calls GET /api/results/:roundId on demand.
- Has a round selector (round-1, round-2, final).
- Maps API rows (restaurant_id, total_score, votes_count) into a typed AggregatedResult.
- Ensures all known restaurants appear, even if they have 0 votes (fills missing with zeros).
- Sorts results by score descending.
- Shows:
 - Leader card (winner with score and votesCount).
 - Full ranking list.

This demonstrates:

- Using fetch in a “tooling” page just for internal use.
- Proper TypeScript typing for API responses (no any).
- Simple admin tooling without extra auth (just don’t share the URL).

8. Resetting Votes on Each Deploy

- In lib/db.ts, ensureSchema() truncates the votes table when VERCEL_ENV === “production”.
- This means every new deployment starts with a fresh poll, while local dev keeps its data.

Lesson:

- Environment-aware behavior based on process.env (development vs production).
- How to implement a simple “reset on deploy” without complex migrations.

9. TypeScript Discipline

Throughout the project:

- Avoid use of any.
- Define small, precise types:
 - ApiResultRow, ApiResponse, AggregatedResult, Restaurant, etc.
- Strong typing of:
 - Route handlers.
 - API client code (fetch + response parsing).
 - Component props and local state.

Why this matters:

- Fewer runtime surprises.
- Better editor autocomplete and refactoring safety.
- Clearer mental model of data shapes.

10. Takeaways / What to Focus on Learning

- Next.js App Router basics:
- File-based routing: `app/page.tsx`, `app/admin/page.tsx`, `app/api/.../route.ts`.
- Building and calling API routes:
- `NextRequest` / `NextResponse`.
- Parameter handling for dynamic routes.
- Connecting to a Postgres DB from Next.js using `pg`:
- Pool, basic queries, schema creation, upsert.
- React patterns:
- `useState` and `useEffect` for UI, side effects, and hydration-safe logic.
- Controlled components (slider) and UI gating (join before voting).
- Project structuring:
- Keep DB logic in `lib/db.ts`.
- UI components in `components/`.
- Admin functionality in its own page.
- Deployment behavior on Vercel:
- Using environment variables (`DATABASE_URL`, `VERCEL_ENV`).
- Resetting state on new deploys.

This project is a complete mini full-stack app:

- Frontend UI with React + Tailwind.
- Backend API with Next.js route handlers.
- Real database with aggregation and upsert logic.
- Simple “admin” tooling and deployment-aware behavior.